

CS201 Lab 6

Data Structures and Algorithms

Shreya Agarwal

Rachit Nimavat

Spring 2026

This lab builds directly upon your your solutions of Stock Price problems from previous labs. If you have not solved those problems, you should do so before starting this lab.

Part 1: Span with Real Stock Prices

Recall Part 1 of Lab 4, where we calculated span for each day. Instead of populating data from user input, we will use real stock prices to test our implementation. Download [BAJFINANCE_day_with_indicators.csv](#) and use that data. You may use `sed` or `awk` to extract the relevant column of stock prices from the CSV file and give it as input to your program with ‘input redirection’ (e.g., `./span < stock_prices.txt`).

Part 2: Rolling Peaks with Real Stock Prices

Recall Part 1 of Lab 5, where we identified rolling peaks in an array. Now, instead of populating data from user input, we will use real stock prices to test our implementation. Download [BAJFINANCE_minute_data_with_indicators.csv](#) and use that data. Set $k = 60$ (minutes instead of days) and identify maximum stock price in every rolling window of 60 minutes. As before, you may use `sed` or `awk` to extract the relevant column of stock prices from the CSV file and give it as input to your program with ‘input redirection’ (e.g., `./rolling_peaks < stock_prices.txt`).

Questions to Ponder Over this Week

1. What is the time complexity of your algorithm for each part in big-O notation? How does it compare to the naive approach? How does it change as you increase the size of the input data? How does it depend on k (the window size in Part 2) and n (the size of the input array)?
2. Empirically verify this by running your program on larger and larger datasets and measuring the time taken. You can use the `time` command in the terminal to measure the execution time of your program (e.g., `time ./span < stock_prices.txt`). Plot the execution time against the size of the input data to visualize how it scales. Similarly, for fixed dataset size, plot the execution time against different values of k to see how it affects the performance of your algorithm.
3. If you are comfortable with Python, you can also implement the same algorithms in Python and compare the execution times with your C implementation. This will give you insights into the performance differences between the two languages for these specific problems.